Exploring Wordplay and Misdirection in Cryptic Crossword Clues with Natural Language Processing

Victoria Winters, Sahana Sundar, Nathan Cantwell, Hans Li

Introduction

Cryptic crossword clues (CCCs) are short phrases that adhere to a strict grammar. Unlike traditional crossword clues that lean on trivia knowledge, CCCs involve letterplay, word substitution, and multiple syntactic/semantic readings of the same text. Each clue contains **a definition** (a word or phrase that approximates the answer), **an indicator** (a word that signals the type of wordplay required), and **fodder** (the raw material manipulated by the wordplay). Crucially, the clue is written so that its surface reading (the apparent meaning of the phrase as a whole) deliberately misleads the reader away from the puzzle's answer.

Related Work: Several researchers have recognized that CCCs are interesting from an NLP perspective (Rozner et al., 2021). Srivastava et al. (2022) included cryptic crossword solving in the BIG-bench benchmark and found it to be one of the tasks that even cutting edge LLMs consistently struggle with. Cleary (1996) and Efrat et al. (2021) highlighted the role of ambiguity in CCCs, and Friedlander & Fine (2018) explored the mental "aha moment" that solvers experience when they see through the misdirection, strikingly akin to the cognitive mechanism theorized about the humor of puns (Hurley et al., 2011). While these works assert that CCC misdirection exists, none have quantified it.

Our project explores this misdirection. We believe CCCs are hard for models because (1) there are many different types of wordplay (wordplay diversity), and (2) each clue is specifically crafted to lead you toward the wrong interpretation (semantic misdirection). Rather than trying to build a solver, which is what most prior work focuses on, we break the problem into two parts:

1. **Can we discover a taxonomy of wordplay types from the data?** We cluster 12,622 unique indicators to see if they naturally group into meaningful wordplay categories.
2. **Can we measure how much misdirection the clue context creates?** We perform a retrieval analysis (answer from definition) and create three classifiers that distinguish between real and fabricated (definition, response) pairs, comparing performance with and without the clue context. This component of the project more generally explores techniques and considerations for creating synthetic data near a model's decision boundary.

In indicator clustering, agglomerative clustering at k=10 produces the most interpretable grouping, with each cluster averaging 65% dominance by a single wordplay type, though no configuration fully recovers a clean taxonomy from the embeddings alone. In the misdirection analysis, our retrieval model provides direct evidence that clue context misleads: embedding a definition within its clue's surface text degrades median answer rank from 1,015 to 2,160 out of 45,254 candidates, a 43% relative drop in top-10 retrieval accuracy. Supervised classification finds that the Random Forest model most successfully discriminates between real and distractor pairs, although interpreting performance and feature importance may speak more to the methods used for dataset creation than the presence of misdirection in CCCs.

Data Source

Our project utilizes the dataset of 660,613 cryptic crossword clues created by George Ho and made available under the Open Database License. The data can be explored online or downloaded (Ho, 2022). It was originally created by scraping clue text, answer, and commentary from nearly a dozen cryptic crossword blogs between January 1, 2009 and July 15, 2023. From the SQLite database, we use variables from two tables:

- Clues Table
  - clue_id: A unique identifier for each clue
  - clue: The full cryptic crossword clue text, including specification of answer format
  - answer: The correct answer
  - definition: The “definition” of the answer, appearing at the start or end of the clue
- Indicators Table
  - indicator_id: A unique identifier for each indicator
  - indicator: Word(s) within the clue that signal the wordplay required to solve it
  - clue_ids: All clue_id values for clues where that indicator appears
  - wordplay_type: One of eight categories which includes anagram, reversal, hidden, container, insertion, deletion, homophone, alternation

We use different subsets, data cleaning processes, and feature engineering for the two parts of this project, and as such, from here we divide the report into two sections: Indicator Clustering and Clue Misdirection. We discuss indicator clustering first, as its results may be relevant when exploring clue misdirection. We conclude this report with a discussion of both sections, what they mean for the structure of cryptic crossword clues.

Indicator Clustering

Feature Engineering: From Raw Data to 10D UMAP Embeddings

Verifying Unique Indicators

Indicators appear in a cryptic crossword clue as intact words or phrases. We leveraged this fact to confirm that all indicators in the Indicators Table were correct. Since the Indicators Table was derived not from the clue itself but from parsing a blogger’s commentary about the clue, verifying that the indicator appeared fully intact within the text of at least one corresponding clue in the Clue Table provided a crosscheck. Indeed, imposing this requirement eliminated indicators that we had first flagged as malformed through other means: unreasonable letter counts, misspellings (PyEnchant), omission from a curated lexical database (WordNet), or extremely uncommon in the English language corpus (Zipf frequency).

Cleaned Dataset Size

This verification reduced our dataset from 15,735 to 14,195 unique (indicator, wordplay) pairs drawn from 70,959 different clues. Because the same indicator string can refer to multiple wordplay types (e.g., "about" functions as a container, reversal, and anagram indicator), these 14,195 pairs correspond to only 12,622 unique indicator strings. The 12,622 unique strings are the input to embedding generation described below. For extrinsic evaluation, we also retain the full mapping of indicators to clues and labels: 76,015 rows, each representing one (clue, indicator) pair with both Ho and–when it could be computed–Ground Truth wordplay labels (See Appendix).

Embedding of Indicator Phrases

We embed each unique indicator string using the BAAI/bge-m3 model from the sentence-transformers library (Chen et al., 2024), producing one 1024-dimensional dense vector per indicator. We do not apply stemming or lemmatization before embedding. Instead, variants like "contribute to," "contributes to," "contributing," and "contribution from" are retained as separate indicators because they may carry distinct grammatical or semantic roles within a clue. The embedding model captures their semantic similarity by assigning them to nearby vectors in the embedding space without requiring explicit normalization. The resulting feature matrix is 12,622 (embeddings of unique indicator phrases) × 1024 (dimensions).

Dimensionality Reduction

In very high dimensionality, distance between points will become computationally expensive and too uniform to meaningfully cluster. To resolve this, we compared results of both PCA and UMAP methods for dimensionality reduction, with 10 dimensional UMAP outperforming PCA. Even at 100 dimensions, PCA only explained 22% of the original variance, and by visual inspection of 2D reduction UMAP is able to capture distinct structural features better than PCA.

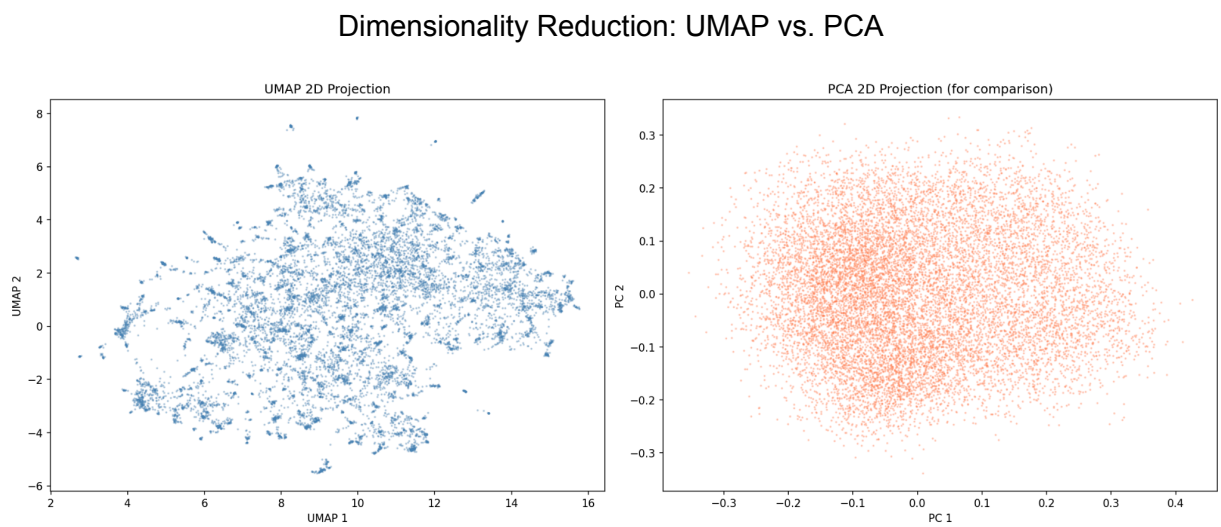

Figure 1: Comparison of UMAP and PCA dimensionality reduction techniques. UMAP demonstrates better density variation of indicator embeddings.

Unsupervised Learning Methods

We selected HDBSCAN and agglomerative clustering with Ward's linkage for our unsupervised learning task. HDBSCAN is a density-based algorithm that discovers clusters of varying density without requiring a pre-specified number of clusters, labeling points in sparse regions as noise rather than forcing assignments. This suits our indicator embeddings, which form a single connected mass with pockets of varying local density; some indicators are nearly identical words or phrases (e.g., "contributing" and "contribution to") while others are only loosely related. Since there is no definitive consensus on wordplay types within the CCC community, HDBSCAN's ability to reveal density structure without imposing a fixed taxonomy is a key advantage. Agglomerative clustering with Ward's linkage complements HDBSCAN by producing a complete clustering tree that can be cut at any level, from a few broad groups to hundreds of

fine-grained ones. Ward's method produces compact clusters, avoiding the elongated chain effects of single-linkage. By sweeping across a range of k values, we can compare clusterings aligned with proposed wordplay taxonomies or discover alternative groupings. K-means was not appropriate for this task, since it assumes spherical, equal-variance clusters. By contrast, we observed an irregular density structure in our 10-dimensional UMAP embeddings.

HDBSCAN

We set min_cluster_size to 10 to capture fine-grained conceptual groupings, and performed a sensitivity analysis over thirteen epsilon values drawn from the first quartile of sampled pairwise distances in the 10D UMAP space. Epsilon controls the distance threshold at which clusters merge, and results proved highly sensitive to this parameter.

Agglomerative Clustering

Since we do not know what granularity best fits the data, we took a two-part approach. First, we swept across 15 k values from k=4 (coarse) to k=250 (fine), tracking silhouette, Davies-Bouldin, and Calinski-Harabasz scores. Second, we selected k=8, 10, 34, and 250 for scatter plots and qualitative inspection. Some of these values correspond to expert-defined wordplay frameworks (e.g., k=8 matches the Ho dataset's labeled types), but the primary goal was to understand the data's inherent structure rather than test a specific taxonomy.

Unsupervised Learning Evaluation

We primarily used silhouette score to evaluate our cluster analysis as this metric incorporates both cohesion within a cluster and separation from other clusters. Additionally, we used the Davies-Bouldin index to understand the extent to which clusters are separate entities from each other. As we have the labelled indicator wordplay types in the dataset, we also looked at average cluster purity to understand the composition of each cluster.

HDBSCAN Epsilon Sensitivity

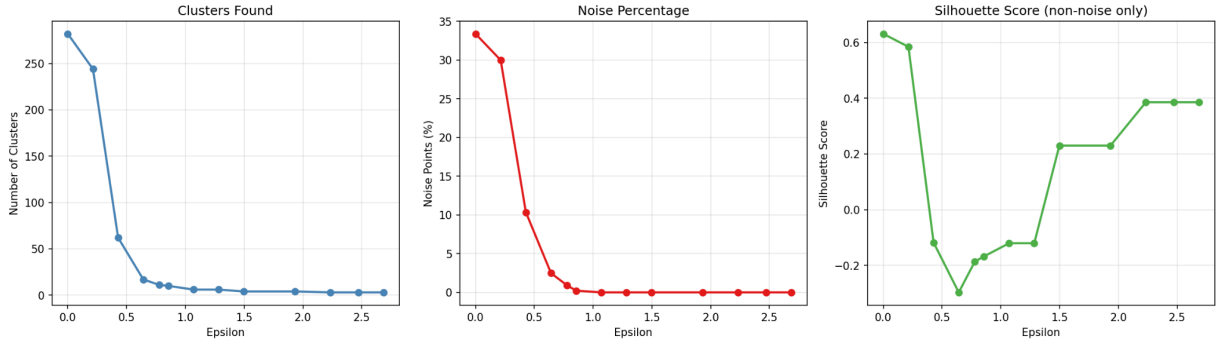

Figure 2: Evaluation metrics for HDBSCAN epsilon sensitivity analysis. Silhouette score is greatest at eps=0, however this forms an overwhelming amount of clusters (281).

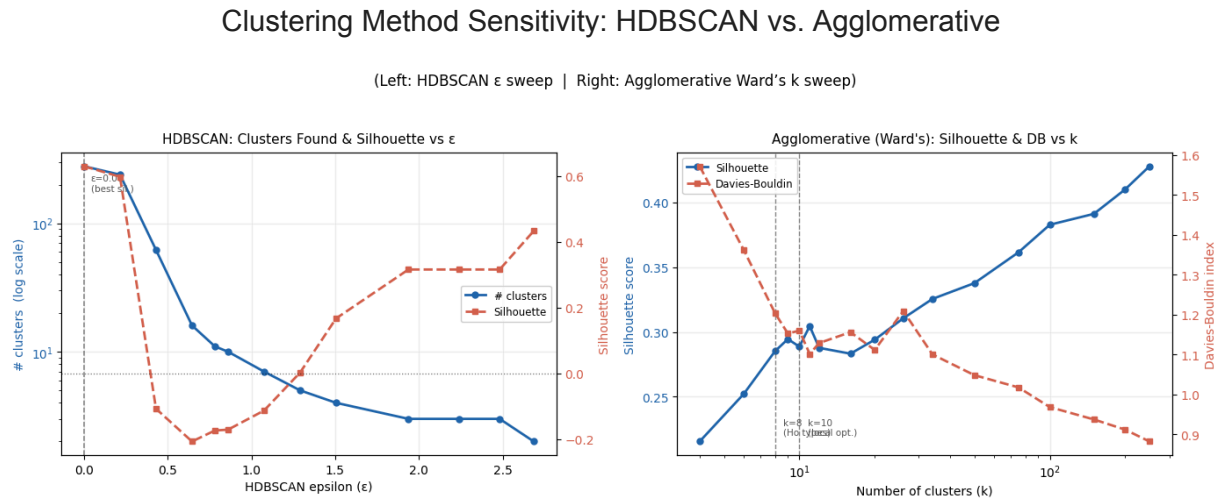

Figure 3: Model Comparison: HDBSCAN shows greater performance at maximum granularity, while agglomerative clustering has sustained performance at moderate levels of granularity.

For the HDBSCAN algorithm, the silhouette score is sensitive to the epsilon value. Silhouette score, noise percentage and the number of clusters formed decrease rapidly between 0-0.65. Lower values of epsilon had high silhouette scores and created many small, fine grained clusters. For example, an epsilon value of 0 resulted in a silhouette score of 0.630, and 281 clusters, many of which individually aligned strongly with specific wordplay types. However, this iteration threw out ~33% of the data as noise, so the high silhouette score only reflects two thirds of the data that's easier to separate. Higher values of epsilon around 2-2.5 also had promising silhouette scores around 0.3, but they only created 2-3 clusters, limiting the discernibility between them. Overall, the clusters formed with the HDBSCAN algorithm didn't provide interpretable groupings that align with existing indicator wordplay categorizations.

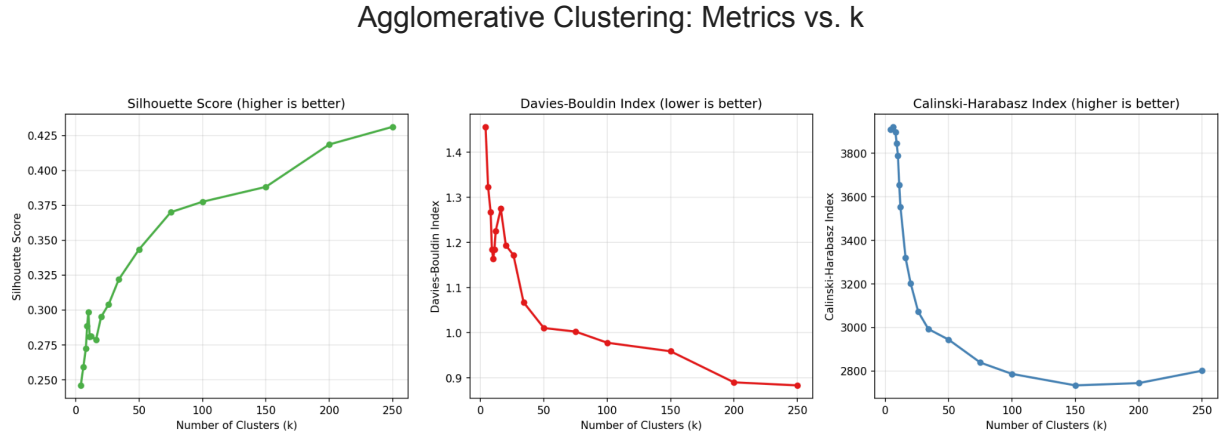

Figure 4: Evaluation metrics for agglomerative clustering sensitivity analysis. k=10 appears promising with a local maximum of 0.299 silhouette score.

For the agglomerative clustering algorithm we see a similar relationship between number of clusters and silhouette score. With 250 clusters there is a silhouette of 0.432, lower than 0.630 for HDBSCAN. For fine-grained clusters agglomerative does not perform as well. However, agglomerative allows us to inspect specific cluster counts, with particular interest at k=10 where there is a local maximum of 0.299 silhouette score. This also corresponds to a local minimum Davies-Bouldin index of 1.164, and a Calinski-Harabasz index of 3789.

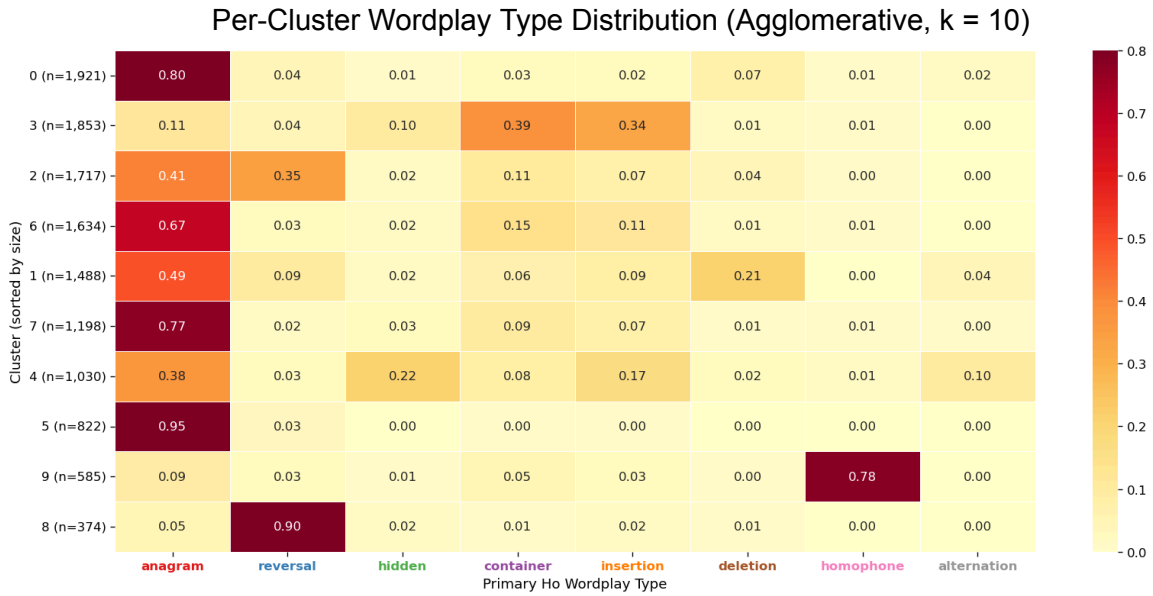

Figure 5: Per-Cluster Ho Type Distribution. The heatmap values correspond to the concentration of each wordplay for the 10 clusters.

In comparing these 10 clusters with the labeled wordplay types of their members, each cluster is on average comprised 65% by its dominant wordplay type. 51% of the dataset are anagram clues, which are the majority type for 7 of the clusters. Reversal and homophone indicators separate well into clusters 8 and 9, respectively. Container and insertion indicators see major overlap with each other, and concentrate together into cluster 3. This implies their embeddings are similar to each other, while being distinct from the embeddings of other wordplay types.

Dendrogram of Cluster Centroids (Agglomerative, k = 10)

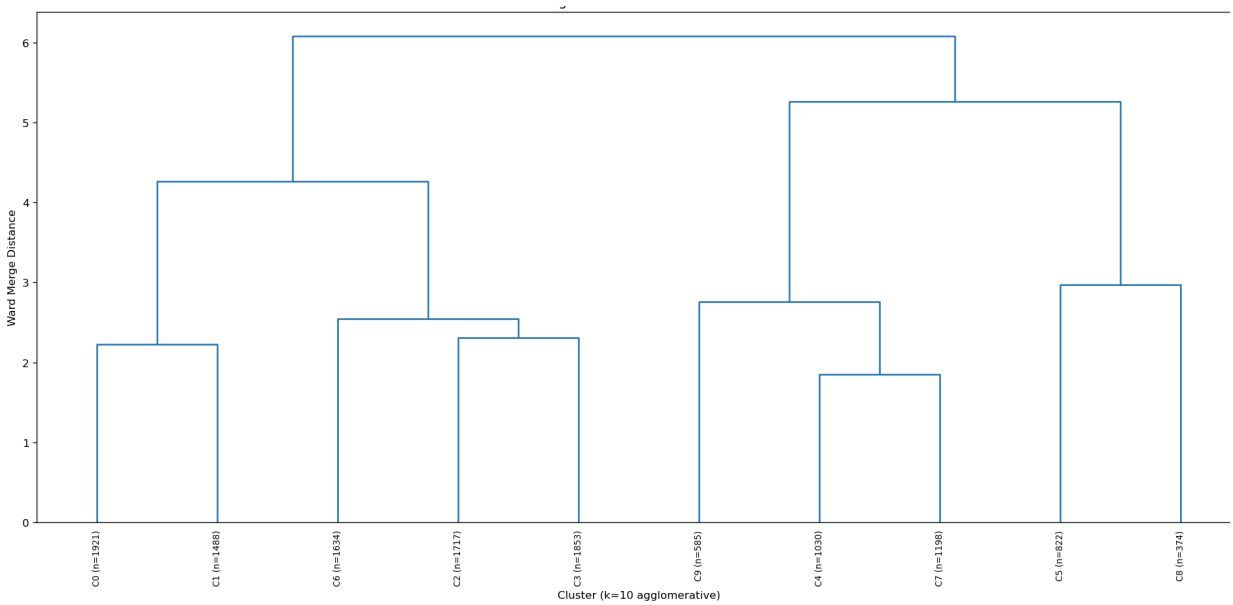

Figure 6: Agglomerative clustering dendrogram for k=10, most merges occurring within Ward's Distance range of 1.8-2.5.

The dendrogram for the merging of these 10 clusters shows the earliest merges for C4+C7, and C0+C1. C4 is varied: anagram, hidden, insertion and C7 is 77% anagram. C0 is 80% anagram. C1 is 49% anagram but also has the highest concentration of deletion at 21%. It appears that the other wordplay types will first merge with an anagram majority cluster before clusters of another wordplay type.

Unified Metrics Comparison Across All Clustering Runs

Figure 7: Silhouette score, Davies-Bouldin index and average cluster purity across all clustering runs. HDSBCAN with epsilon=0 and Agglomerative clustering with k=8, k=10 and k=34 are highlighted as key runs

Looking across all clustering runs, HDBSCAN tends to have higher clustering purity than agglomerative clustering but lower silhouette scores. Additionally, constrained clustering didn't provide a significant improvement across any of the metrics.

Misdirection in Cryptic Crossword Clues

Feature Engineering: From Raw Data to 47 Features

Clue Preprocessing

The raw dataset contained 660,613 clues. We applied seven filter steps that leveraged rules from CCC construction to verify that blog content was parsed correctly: (1) removed rows with missing definition, answer, or clue fields; (2) removed mis-parsed bracketed clues; (3) validated the answer's length and format against the clue's parenthetical length indicator; (4) split double-definition clues into one row per valid definition; (5) verified each definition appears as an intact, word-bounded substring in the clue's surface text; (6) verified each definition is positioned at one edge of the surface (start or end), as cryptic grammar requires; and (7) required both definition and answer to have at least one WordNet synset. Definitions and answers may be multi-word as long as they appear in WordNet.

These filters reduced 660,613 raw clues to 241,397 filtered rows covering 129,429 unique (definition, answer) pairs: an average of 1.9 clues per pair, reflecting puzzle setters' habit of reusing the same definition-answer relationship for different clues. After CALE phrase construction (described below), which drops a further 1,186 rows where the delimiters could not be placed unambiguously, the final working dataset is 240,211 rows.

Embedding Generation

Cryptic crossword clues exploit polysemy: the surface reading activates a different sense of the definition word than the one that points to the answer. To measure this, we need an embedding model that produces genuinely distinct representations for a word-in-context versus that same word in isolation. Sentence-transformer models can (theoretically) satisfy this constraint because the underlying transformer produces contextualized token-level representations, and the representation for a given target word can be extracted before the model's pooling step collapses all tokens into a single sentence vector. By comparison, Concept-Aligned Embeddings (CALE) (Loiseau et al., 2024) are specifically designed to disambiguate word senses in context using delimiter tags around a target word (or phrase) to tell the model where to focus. We compared the performance of two sentence-transformers and one CALE model to create contextualized word embeddings that are distinct from *both* the decontextualized word *and* the context sentence. The standard sentence-transformer models (all-mpnet-base-v2, bge-base-en-v1.5) do not meet our requirements: while they differentiate nearly as well as the CALE model between a decontextualized and contextualized word, the contextualized word embedding is effectively identical to the embedding for the full sentence context, which leads us to question the appropriateness and interpretability their contextualized word embeddings.

We selected `gabrielloiseau/CALE-MBERT-en`, a 1024-dimensional ModernBERT-based model designed to disambiguate word senses in context. We generate seven embeddings per (clue, definition, answer) row (see Appendix):

1. **Allsense**: (definition and answer) Average of all embeddings of WordNet meanings.
2. **Common**: (definition and answer) The most common WordNet meaning.
3. **Obscure**: (definition and answer) The least common WordNet meaning.
4. **Clue Context**: (definition only) The meaning of the target as it appears in the clue.

The allsense construction first creates unique embeddings of the target word for each of its WordNet synsets, then averages the resulting vectors. This approximates a sense-grounded "decontextualized" representation while staying within CALE's training distribution. Feeding the model a bare word with no context produced embeddings that failed to discriminate related from unrelated word pairs in preliminary testing. Preferentially, we constructed CALE phrases from the synset usage example (22% of all targets) or unaltered definition text (3%). When the word does not appear in the synset's own text (75%), we fall back to the format `<t>word</t>: synset_definition_text`.

Approximately 35% of definitions and 46% of answers have only a single usable WordNet synset. For these words, the allsense, common, and obscure embeddings are identical by construction.

Three Feature Families Linking Definition and Response

We create three families of features that contain 47 features in all.

1. **Meaning features** (21 total) - cosine similarities between CALE embeddings, split into two subgroups:
    - Context-free meaning (15): cosine similarities among the 6 context-free embeddings (allsense, common, obscure for both definition and answer)
    - Context-informed meaning (6): cosine similarities between the clue-context embedding and each of the 6 context-free embeddings
2. **Relationship features** (22) - WordNet graph-based: 20 boolean two-hop relationship types, max path similarity, and shared synset count
3. **Surface features** (4) - string-level: edit distance, length ratio, shared first letter, character overlap ratio

Context-free meaning features are removed when running the models on the harder dataset to avoid data leakage, since they were used to select distractors. The remaining context-informed features–capturing how clue context shifts the definition embedding–are toggled between A and B conditions to isolate the misdirection effect. The retrieval analysis draws on both meaning and relationship features, (1) ranking candidate answers by cosine similarity across embedding conditions and (2) measuring WordNet reachability via synonym, hyponym, and other paths.

Overall Methodology

We approach the misdirection question from two complementary angles. The retrieval analysis measures the effect directly: for each of the four definition embeddings (allsense, common, obscure, and clue context), we rank all 45,254 candidate answers by cosine similarity, and record where the true answer falls. Clue-context misdirection, if real, shows up as a worsening of rank when the definition is embedded inside the clue surface. We also completed the classification analysis with trained supervised models: we construct balanced binary datasets of real versus distractor (definition, response) pairs, describe each pair with 47 engineered features, and train three model families with and without the six clue-context features. This gives a second, multivariate lens on the same question, though as we discuss under Tradeoffs its relationship to the retrieval finding is not straightforward.

Retrieval Analysis

Word Meanings: Retrieval Using Cosine Similarity of CALE Embeddings

The retrieval analysis runs over the 127,608 unique (definition, answer) pairs in our filtered dataset, using the 45,254 unique answers as the candidate pool. We report over unique pairs rather than all 240,211 clue rows because the same pair can appear in many different clues, using all rows would count a pair's context-free ranks (which are identical across clues with the same definition) multiple times, artificially inflating the apparent gap between context-free and context-informed conditions. For the clue-context condition, where each clue gives a different embedding, we take the median rank across all clues for each pair.[1]

Embedding the definition within the clue's misleading surface text degrades median rank of the allsense answer from 1,015 to 2,160 (+1,145 positions, out of 45,254 candidates) relative to the sense-averaged baseline. Top-10 retrieval accuracy falls from 7.69% to 4.40%, a 43% relative drop.

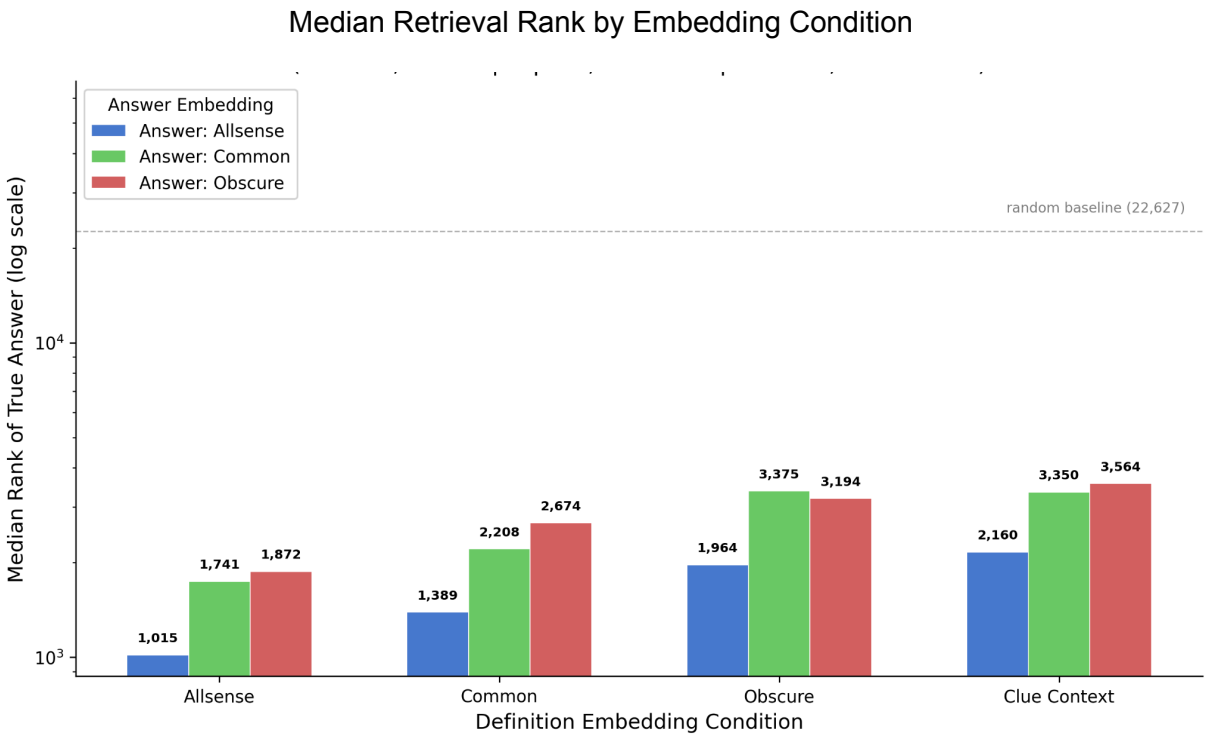

Figure 8: Median rank of the true answer (log scale) across four definition-embedding conditions and three answer-embedding conditions, over 127,608 unique pairs with 45,254 candidates. The Clue Context group is the worst or second-worst definition condition regardless of the answer condition.

[1] An all-rows analysis (240,211 clue rows using the allsense condition only) gives a median rank of 831 context-free, somewhat lower than the unique-pairs 1,015, confirming that frequently-reused (definition, answer) pairs tend to be easier and validating unique pairs as the more conservative reporting unit.

Allsense consistently outperforms both Common and Obscure on both the definition and answer sides–averaging across senses hedges against committing to the wrong one. Surprisingly, committing to the obscure sense of the definition (Obscure Def × Common Ans: median rank 3,375) can degrade retrieval *more* than clue-context misdirection does (Clue Context Def × Allsense Ans: 2,160). This could reflect the overall difficulty of the clue: puzzle creators who use obscure meanings for definition are more inclined to use an obscure meaning for the answer. Wrong-sense commitment and surface misdirection are different mechanisms–the first picks a wrong meaning outright, the second is misled by surrounding text–but they produce comparable retrieval damage.

Word Relationships: Retrieval Using WordNet Connectivity

The WordNet Connectivity analysis measured how many of the 127,608 unique (definition, answer) pairs are structurally reachable in WordNet's taxonomy by traversing hypernym, hyponym, meronym, holonym, and synonym edges up to 3 hops deep. The analysis found that only about 52% of pairs are reachable within 3 hops, with 11.5% connected by synonymy, 30.3% within 1 hop, 44.1% within 2 hops, and an additional 7.6% gained at 3 hops. Three relationships dominate the connected pairs: co-hyponymy (~20%), direct hyponymy (~15%), and synonymy (~12%), while part-whole relationships are rare (see Appendix).

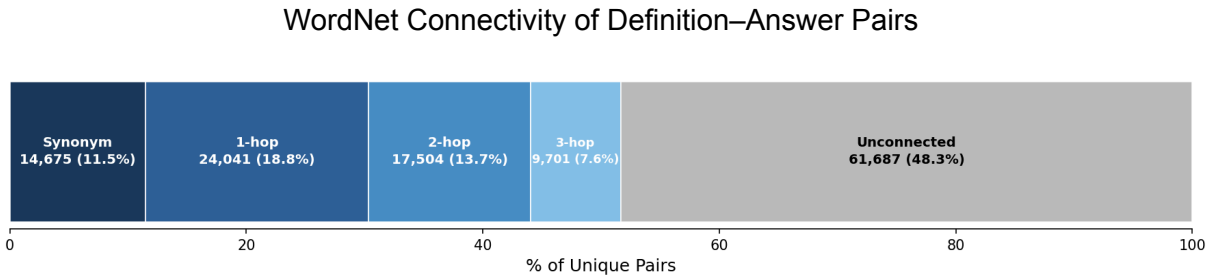

Figure 9: Cumulative reachability of definition–answer pairs in WordNet by number of hops. Roughly 52% of pairs are reachable within three hops; the remainder have no taxonomic path.

Combining Word Meaning and Relationship

Retrieval performance improves with tighter WordNet connectivity, from synonyms (best) through 1-hop, 2-hop, 3-hop, to unconnected pairs (worst). This confirms that structural proximity in WordNet’s taxonomy tracks distributional proximity in our chosen embedding space

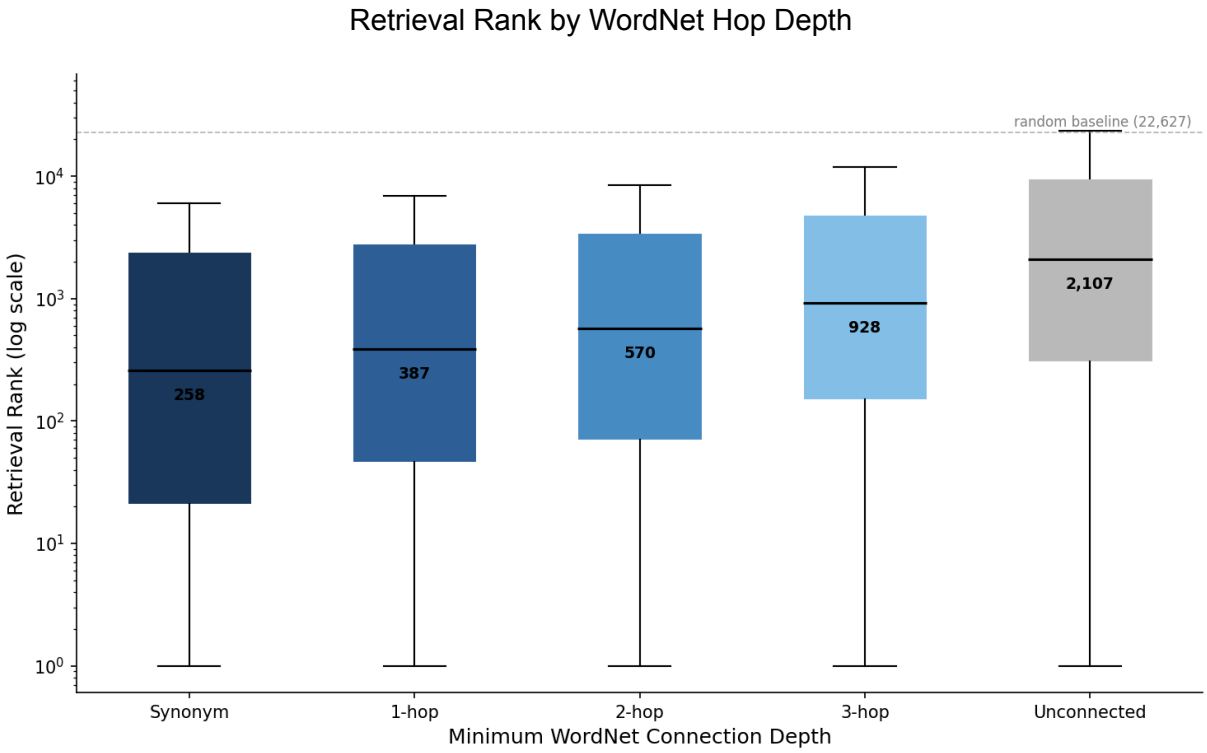

Figure 10: Median retrieval rank by WordNet hop depth. Pairs with tighter taxonomic connectivity are retrieved more accurately in both the context-free and clue-context conditions.

The misdirection effect is present for both groups but far more pronounced for connected pairs (median delta 763, 64.4% worsened) than unconnected pairs (median delta 4, 50.2% worsened), suggesting that clue context most effectively disrupts retrieval when there is an obvious taxonomic link between definition and answer.

Supervised Learning Methods: Three Classification Model Families

The classification task is binary: given a (clue, definition, answer) triple described by its 47 features, predict whether the answer is the real crossword answer or a distractor. We train three model families chosen for mechanistic diversity:

- **K-Nearest Neighbors**: instance-based, non-parametric. Hyperparameters: neighbor count, distance weighting, and metric, tuned via grid search.
- **Logistic Regression**: probabilistic, linear. Elastic-net regularization (grid search over C and l1 ratio) to handle multicollinearity among cosine features.
- **Random Forest**: tree-based, non-linear. Tuned via randomized search over tree count, depth, and split/leaf thresholds. Strongest of the three in every experiment; used for the in-depth evaluation below.

Evaluation uses nested cross-validation: 5-fold GroupKFold on the outside, grouped by (definition, answer) pair so that different clues sharing the same underlying pair cannot leak

across train and test; 3-fold StratifiedKFold on the inside for the hyperparameter search. StandardScaler is fitted only on training folds and applied to the corresponding test fold, and is used only for KNN and Logistic Regression (Random Forest is scale-invariant). All folds use the same random seed so that a given (clue, definition, answer) row sits in the same fold across every experiment. We report mean +/- standard deviation of accuracy across the five outer folds, plus F1, precision, recall, and ROC AUC for all three model families.

Easy and Harder Datasets: Two ways of constructing distractors

Our classifiers easily discriminated between true and distractor pairs in our easy dataset, where the distractor response was chosen at random from the set of 45,254 unique answers. Real pairs average 0.648 cosine similarity and random distractors average 0.430, so all three models achieve 86–88% accuracy regardless of whether clue-context features are included. We keep this easy dataset (480,422 rows, 47 features) as a sanity-check baseline; any large divergence between the two feature conditions here suggests a data leakage issue or feature computation error.

To construct distractors for a harder dataset, we draw each response from the top-100 most cosine-similar candidates to the definition (using the allsense embeddings). These near-miss distractors appear more plausible to human CCC solvers, but more importantly for our classifiers, false responses occupy nearly the same region of embedding space as the true answer, so the context-free cosine features can no longer separate the classes and the classifier must rely on subtler signals.[2] Since the distractors were selected by Allsense cosine similarity, all 15 context-free cosine features are contaminated by the construction process and are removed from our models. That leaves 32 features for the harder dataset experiments (6 context-informed + 22 WordNet + 4 surface), or 26 when context features are ablated.

Supervised Evaluation

Evaluation Metrics

For the classification experiments, the primary metric is accuracy on held-out GroupKFold test folds (mean ± standard deviation across 5 folds), since the datasets are class-balanced by construction. We also report F1, precision, recall, and ROC AUC for all three model families. Because the outer cross-validation is grouped by (definition, answer) pair, a model never sees a clue-level variation of a pair it has been tested on — this is the stricter leakage regime and gives somewhat lower (and more honest) accuracy than simple stratified splitting would.

Classification Results

Table 3 reports all three model families on the easy and harder datasets (480,422 rows each). Experiment A includes the six clue-context features; Experiment B drops them. Therefore Delta reflects the amount of misdirection: how much does classification improve by disregarding clue context?

---

[2] Under Allsense (context-free) embeddings, real pairs average 0.648 cosine similarity vs. 0.430 for distractors. Under clue-context embeddings (which were not used for distractor selection) real pairs drop to 0.545 while distractors rise to 0.615, flipping and narrowing the gap. Investigating why and how this happens could be a valuable first step in creating a third “Hardest” dataset, where we continue zooming in around the model’s decision boundary.

Model	Easy A (47 feat)	Easy B (41 feat)	Δ Easy*	Harder A (32 feat)	Harder B (26 feat)	Δ Hard*
K-Nearest Neighbors	86.3 ± 0.1%	86.2 ± 0.2%	-0.1pp	73.9 ± 0.1%	64.5 ± 0.3%	**-9.4pp**
Logistic Regression	86.9 ± 0.2%	86.7 ± 0.2%	-0.2pp	72.1 ± 0.2%	66.6 ± 0.3%	**-5.5pp**
Random Forest	87.7 ± 0.1%	87.3 ± 0.1%	-0.4pp	**75.7 ± 0.1%**	67.3 ± 0.3%	**-8.4pp**

*Table 3: Classifier accuracy with and without clue-context features. *Delta signifies change in model performance from REMOVING context. (5-fold GroupKFold CV, mean ± SD)*

Random Forest is strongest in every condition and is the model we use for the in-depth evaluation below. Standard deviations are small (≤ 0.3pp), and the ordering of models is mostly stable across both datasets and both feature conditions.

The easy-dataset experiment comparison behaves exactly as a sanity check should: absolute Δ ≤ 0.4pp across all models, confirming that with trivially-wrong random distractors the six context features contribute nearly nothing beyond what the 41 context-free features already capture. The harder-dataset experiment column shows a more pronounced effect: Δ ranges from -5.5 to -9.4pp, meaning the six context-informed cosine features noticeably help the classifiers distinguish between real and distractor (definition, response) pairs. This does *not* mean that ignoring clue context makes it harder to solve a CCC (see SHAP below), and may be more of an artifact of how the harder dataset was created than a measure of misdirection in CCCs: models run on the harder dataset removed context-free meaning features (which were used to select distractors), so context-informed features become relatively more important by design, not necessarily because of misdirection.

Feature Importance and Ablation Analysis

We analyze feature contributions on the best-performing model (Random Forest, harder dataset, Experiment 2A with all 32 features) using both Gini-based feature importance and a group-level ablation: removing one feature group at a time and re-running 5-fold cross-validation.

Feature group removed	Features removed	Remaining	Accuracy	Change
None (baseline)	—	32	74.0%	—
Context-Informed cosine	6	26	63.2%	**−10.8pp**
WordNet relationship	22	10	65.3%	**−8.7pp**

Feature group removed	Features removed	Remaining	Accuracy	Change
Surface string	4	28	73.4%	**−0.6pp**

Table 4: Group-level ablation — Random Forest, harder dataset, Experiment 2A baseline accuracy 75.8%

The six context-informed cosine features carry more predictive weight per feature than any other group, removing them costs 10.9pp, almost double the per-feature impact of the relationship group (9.8pp drop from removing 22 features). This is consistent with the headline Δ Hard result: on the harder dataset, the context-informed embeddings are meaningfully contributing to distinguishing between real and distractor pairs. The four surface features, by contrast, are nearly dispensable (−1.0pp). Note that the 2A to 2B comparison in Table 3 is itself a group-level ablation of the context features (+8.4pp for RF), the minor numerical difference from Table 4 (−10.9pp) reflects a different fold-0 train/test split versus full 5-fold CV, not a substantive disagreement.

The SHAP beeswarm below confirms that WordNet path similarity is the strongest global driver, with higher values (i.e., definition and response are closer in WordNet's taxonomy) consistently pushing predictions toward the positive class (real pairs). In contrast, one of the key context-informed cosine features (Def Clue Context x Resp Allsense) shows a bidirectional effect: when the clue strongly shifts the embedding toward Resp, the model often interprets this as a distractor-like pattern rather than evidence for correctness. This asymmetry indicates that cosine shifts are not simply "higher is better," but interact with lexical relationship features in structured ways. Such nonlinear, interaction-dependent behavior helps explain why Random Forest outperforms Logistic Regression: the signal is not linearly separable, and tree-based models are better suited to capturing these feature entanglements.

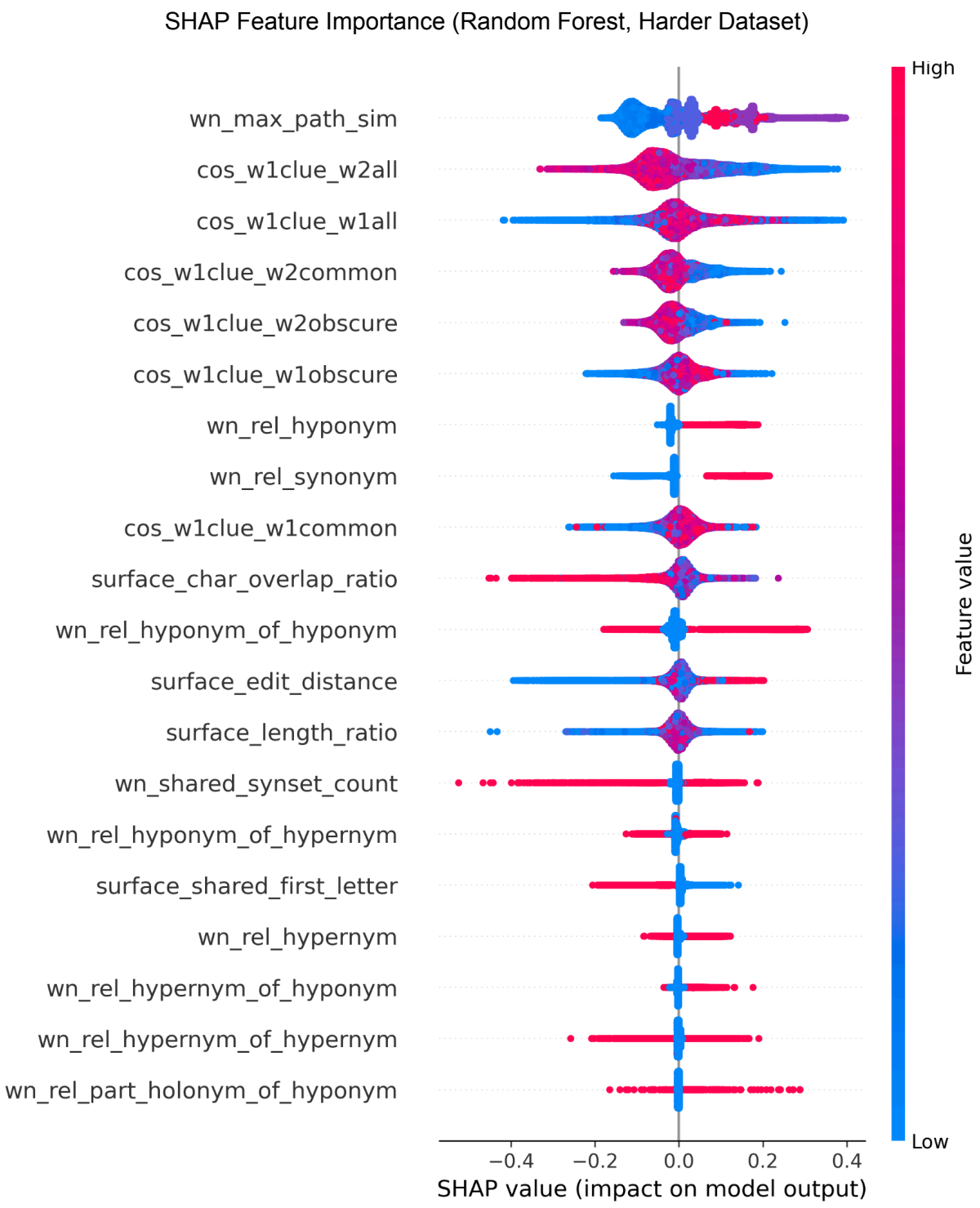

Figure 11: SHAP summary plot for the best classifier on Experiment 2A. Each point is one test example; horizontal position is contribution to the prediction, colour is feature value. The six context-informed cosine features and wn_max_path_sim dominate the top of the ranking.

Sensitivity Analysis

Retraining Random Forest on 10% to 100% of the harder-dataset training fold gives test accuracy that varies by less than 0.5pp across the entire range, so the ceiling is feature-limited rather than data-limited. We also varied the top-k threshold for distractor selection: narrowing to top-20 drops baseline accuracy by about 2pp and widens Δ Hard, while widening to top-500 raises baseline by about 3pp and compresses Δ Hard toward the easy-dataset regime. The direction of Δ Hard is stable across the range, so we report top-100.

Tradeoffs

Retrieval vs. classification framing. Retrieval produces a +1,145-median-rank effect; classification reflects entanglement between model features and methods used to construct the easy and harder datasets. Retrieval is the more direct measurement of clue misdirection, but the dataset creation process is more universally interesting: constructing increasingly convincing distractors has the potential to reveal interesting patterns in natural language and identify what, precisely, is so uniquely difficult about CCC for LLMs.

Interpretability vs. accuracy. Logistic Regression gives readable coefficients but is weakest (72.1%); Random Forest is strongest (75.7%) but less interpretable. We use RF for importance ranking and LR coefficients as a directional sanity check.

Failure Analysis

Of the 23,244 misclassified examples from Random Forest on the harder test fold, most fall into three overlapping categories. The categories overlap heavily, and 84.3% of errors fall into at least one.

Semantic near-miss (57.3%). The (Clue Context Def x Allsense Resp) cosine similarity is atypical for the true class, with two distinct sub-patterns. For false positives, a distractor pair has unusually high cosine similarity (above the real-pair median of 0.4661), making it look like a real pair. For false negatives, a real pair has unusually low cosine similarity (below the distractor median of 0.6003), making it look like a distractor. The FN direction dominates: the more common failure mode is when real pairs don't look semantically close enough.

Surface artifact (50.7%). While string surface features are *not* powerful misleaders themselves (see ablation results), in cases where semantic features are ambiguous, the weak signal from surface features can tip the model's prediction the wrong way. The FP/FN split shows that false negatives dominate: the model rejects real pairs with low surface overlap more often than it accepts distractors with high surface overlap.

Polysemy confusion (20.4%). The definition has an unusually large number of synsets (> 75th percentile of the test set), meaning its Allsense embedding is pulled in many directions.

Below are three specific data records, selected by these criteria across all ~23,244 misclassified examples:

1. **"flier"** → **"airman"** — Most confident false positive (P(real) = 0.9770). This is a distractor pair the model was almost certain was real, because wn_max_path_sim = 1.0; "flier" and "airman" are essentially synonyms in WordNet, making it genuinely hard to distinguish from a real pair.
2. **"furnish"** → **"endue"** — Most confident false negative (P(real) = 0.0675). This is an actual real pair the model was very confident was a distractor. 7 of its 10 top features were closer to the distractor class mean than the real class mean.
3. **"cheer"** → **"alarming"** — Near decision boundary (P(real) = 0.5001, essentially a coin flip). This is a distractor the model barely misclassified as real. "Cheer" has 7 synsets (high polysemy), and 6 of its 10 top features fell closer to the wrong class mean.

Fixes: Semantic near-misses need features and dataset construction that better capture human word use (possibly achieved by using a more comprehensive/modern dictionary in the embedding pipeline or adding new meaning features that employ a cross-encoder to produce an alternative similarity score for definition and response); surface artifacts suggest dropping the string features entirely (only about 1pp accuracy loss per the ablation) or using L1 regularization or recursive feature elimination to reduce reliance on surface features when semantic features are sufficient; polysemy confusion suggests revisiting how Allsense embeddings are computed or defining new categories of embeddings entirely (potentially using a Word Sense Disambiguation model).

Discussion

Indicator Clustering. It is not surprising that indicator embeddings do not cleanly separate into wordplay types, but it did surprise us that the dataset creator's eight labeled wordplay categories do not emerge as a distinctive structure at any level of granularity. Even at k=8, the dominant wordplay type (anagram) is distributed across nearly all clusters rather than concentrating into one, and constrained clustering with expert-selected seed words (Tiernan & Runnalls, 2025) did not significantly improve alignment. A related challenge was that HDBSCAN consistently preferred far more clusters than the eight labeled types: at epsilon=0, it produced 281 clusters, many of which were tight groupings of near-synonymous phrases (e.g., slight morphological variants of the same word). This suggests that before meaningful wordplay-level structure can emerge, the data cleaning, embedding, or clustering pipeline needs a mechanism for collapsing near-duplicate indicator phrases. Addressing this would also let us revisit a question about the indicators we excluded as literal duplicates across wordplay types: is the dominant wordplay type truly the most common form of wordplay, or does it simply have the greatest diversity of indicator phrases? HDBSCAN's best silhouette scores came at the cost of discarding roughly a third of the data as noise, and configurations that retained all data produced far weaker separation. With more time, we would explore whether indicators cluster more naturally by the type of operation they describe (e.g., selecting letters, rearranging letters, combining chunks) or by the conceptual metaphors they invoke (e.g., concealment, movement, destruction), as these alternative taxonomies are reminiscent of the embodied metaphors that underpin mathematical thinking (Lakoff & Núñez, 2000).

Clue Misdirection. The retrieval analysis produced the clearest surprise: committing to the wrong word sense (obscure definition × common answer) can degrade retrieval nearly as much

as clue-context misdirection, suggesting that wrong-sense commitment and surface misdirection are distinct mechanisms that produce comparable damage. A major challenge was embedding model selection. Standard sentence-transformer models initially appeared viable, but we discovered that their contextualized word embeddings were effectively identical to their full-sentence embeddings, making them unsuitable for isolating the effect of clue context on a specific target word. Switching to CALE resolved this but required engineering a WordNet-based pipeline to construct sense-grounded input phrases, since the model could not produce meaningful embeddings from bare words. A second challenge was interpreting the supervised classification results: because the harder dataset removed context-free cosine features (which were used to select distractors), the remaining context-informed features became relatively more important by design, making it difficult to attribute the accuracy delta purely to misdirection. With more time, we would construct an alternative harder dataset using WordNet relationship matching rather than cosine similarity to select distractors, which would allow all cosine features to remain in the model and provide a complementary experimental condition. We would also train a pointwise retrieval model to more directly bridge the gap between the retrieval and classification analyses, and break down misdirection effects by wordplay type.

Ethical Considerations

The ethical considerations for this project arise from the initial dataset itself. First, a potential issue of data provenance and secondary use. While the George Ho dataset is under open license, it was created through web scraping of hobbyist blogs and forums. We have not verified whether the scraping violated each websites' terms of use at the time of scraping. The original blog posters were simply CCC enthusiasts who would not reasonably expect their comments to be used in this way. While it would not be feasible to obtain individual consent from each anonymous user, a possible mitigation strategy would be to notify the administrators of each forum and make the project available for users to review the use of their data. Second, the dataset is entirely in English which introduces a bias of only serving an English speaking audience. Serving only English speakers could be viewed as a case of cumulative disadvantage, on a global scale English proficiency is a marker of access to educational and economic resources. Those without English proficiency and its correlated benefits would also find this analysis inaccessible.

Statement of Work

Victoria (CCC Domain Expertise): Research questions, project management, embeddings, AI-assisted notebook refactoring and repo management
Sahana (Evaluation): Evaluation and visualization, supervised/unsupervised support as needed
Nathan (Unsupervised Learning): Indicator clustering, definition clustering comparison
Hans (Supervised Learning): Clue misdirection, AI prompting support
All: Data cleaning, report writing, intermittent reporting to faculty advisor

References

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). *BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through*

self-knowledge distillation (Version BAAI/bge-m3) [Computer software]. Hugging Face. https://huggingface.co/BAAI/bge-m3

Cleary, J. (1996). Misleading contexts: The construction of ambiguity in the cryptic crossword clue. *Edinburgh Working Papers in Applied Linguistics, 7*, 14-29.

Efrat, A., Shaham, U., Kilman, D., & Levy, O. (2021). Cryptonite: A cryptic crossword benchmark for extreme ambiguity in language. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 4186– 4192, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.emnlp-main.344

Friedlander, K. J., & Fine, P. A. (2018). “The penny drops”: Investigating insight through the medium of cryptic crosswords. *Frontiers in psychology*, *9*, 904. https://doi.org/10.3389/fpsyg.2018.00904

Ho, G. (2022). A Dataset of Cryptic Crossword Clues. https://cryptics.georgeho.org/

Hurley, M. M., Dennett, D. C., & Adams, R. B., Jr. (2011). *Inside jokes: Using humor to reverse-engineer the mind*. MIT Press.

Lakoff, G., & Núñez, R. E. (2000). *Where mathematics comes from: How the embodied mind brings mathematics into being*. Basic Books.

Liétard, B., & Loiseau, G. (2025). CALE: Concept-aligned embeddings for both within-lemma and inter-lemma sense differentiation (arXiv:2508.04494). arXiv. https://doi.org/10.48550/arXiv.2508.04494

Rozner, J., Potts, C., & Mahowald, K. (2021). Decrypting cryptic crosswords: Semantically complex wordplay puzzles as a target for nlp. *Advances in Neural Information Processing Systems*, *34*, 11409-11421. https://doi.org/10.48550/arXiv.2104.08620

Srivastava, A., Siro, C., Shutova, E., Jumelet, J., ter Hoeve, M., Giulianelli, M., Lewis, M., Schubert, M., Tong, X., & BIG-bench authors (2022). Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. (v2 ed.) ArXiv. https://doi.org/10.48550/arXiv.2206.04615

Tiernan, A., & Runnalls, L. (2025). *Minute cryptic: Over 160 wordplay puzzles to decipher, unlock, and untangle*. St. Martin’s Griffin.

Appendix A — Ground Truth Wordplay Labels (Indicator Clustering)

The wordplay labels in the Indicator Table were derived by Ho (2022) from bloggers' commentary rather than from the clue text itself. For four of the eight wordplay types, the label can be algorithmically verified by checking whether the answer's letters can be recovered from the clue text through the type's defining operation:

Wordplay type	Verification operation
Hidden	The answer appears as a contiguous substring of the clue surface text
Reversal	The reversed answer appears as a contiguous substring of the clue surface text
Alternation	The answer appears in every-other-letter positions of a substring of the clue surface text
Anagram	A substring of the clue surface text is a letter-permutation of the answer

Because of advanced cryptic mechanics such as charades (where the answer is assembled from pieces that individually may not match the operation), these algorithmic checks do not perfectly align with the blogger-derived labels. Where both exist for the same clue, they agree 92.6% of the time. We therefore retain both label sets: the blogger-derived "Ho Labels" (more comprehensive, covering all eight types, but subjective) and the algorithmically derived "Ground Truth Labels" (covering four types, objective but narrower). Both are used in the cluster-purity evaluation of Part B.

Appendix B — Embedding Types and Feature Inventory (Clue Misdirection)

B.1 Seven embedding types

Name	What it is	Computed per
word1_allsense	Definition embedded in each of its WordNet synset contexts, then averaged	Unique definition
word1_clue_context	Definition embedded within the clue surface text using CALE's <t></t> delimiters	Clue row
word1_common	Definition embedded in its most-common WordNet synset context	Unique definition
word1_obscure	Definition embedded in its least-common WordNet synset context	Unique definition
word2_allsense	Answer embedded in each of its WordNet synset contexts, then averaged	Unique answer
word2_common	Answer embedded in its most-common WordNet synset context	Unique answer
word2_obscure	Answer embedded in its least-common WordNet synset context	Unique answer

Synset-context phrases are constructed by inserting <t></t> delimiters around the target word within each synset's usage example or definition text, falling back to "<t>word</t>: definition_text" when the word does not appear in the synset's own text. Approximately 35% of definitions and 46% of answers have only one usable WordNet synset, making their allsense, common, and obscure embeddings identical.

B.2 The 47 features and their use in each experiment

Group	n	Features	Easy A (47)	Easy B (41)	Hard A (32)	Hard B (26)
Context-free cosine *Pairwise cosine similarities among the six embeddings that do not involve the clue surface*	15	cos_w1all_w2all, cos_w1all_w2common, cos_w1all_w2obscure, cos_w1common_w2all, cos_w1common_w2common, cos_w1common_w2obscure, cos_w1obscure_w2all, cos_w1obscure_w2common, cos_w1obscure_w2obscure, cos_w1all_w1common, cos_w1all_w1obscure, cos_w1common_w1obscure, cos_w2all_w2common, cos_w2all_w2obscure, cos_w2common_w2obscure	✓	✓	—	—
Context-informed cosine	6	cos_w1clue_w1all, cos_w1clue_w1common, cos_w1clue_w1obscure, cos_w1clue_w2all,	✓	—	✓	—

Cosine similarity between the clue-context definition embedding and each of the other six. These are the features toggled in the A-vs-B experiments.		cos_w1clue_w2common, cos_w1clue_w2obscure				
WordNet relationship *20 boolean two-hop relationship flags, plus maximum path similarity and shared synset count*	22	wn_rel_synonym, wn_rel_hyponym, wn_rel_hypernym, wn_rel_part_holonym, wn_rel_part_meronym, wn_rel_substance_meronym, wn_rel_member_meronym, wn_rel_hyponym_of_hypernym, wn_rel_hypernym_of_hyponym, wn_rel_hyponym_of_hyponym, wn_rel_hypernym_of_hypernym, wn_rel_part_holonym_of_hyponym, wn_rel_hyponym_of_part_holonym, wn_rel_substance_meronym_of_hyponym, wn_rel_part_meronym_of_hyponym, wn_rel_hyponym_of_part_meronym, wn_rel_part_meronym_of_hypernym, wn_rel_part_holonym_of_hypernym, wn_rel_part_holonym_of_part_meronym, wn_rel_member_meronym_of_member_holonym, wn_max_path_sim, wn_shared_synset_count	✓	✓	✓	✓
Surface *String-level comparisons between definition and answer*	4	surface_edit_distance, surface_length_ratio, surface_shared_first_letter, surface_char_overlap_ratio	✓	✓	✓	✓
Total	**47**		**47**	**41**	**32**	**26**

The 15 context-free cosine features are removed from the harder dataset because distractors are selected by allsense cosine similarity, making those features artifacts of dataset construction.

B.3 Additional classification metrics (5-fold GroupKFold CV, mean ± SD)

Experiment	Model	Accuracy	F1	Precision	Recall	ROC AUC
Easy A (47 feat)	KNN	0.857 ± 0.004	0.846 ± 0.005	0.913 ± 0.006	0.789 ± 0.009	0.926 ± 0.001
	Logistic Reg.	0.868 ± 0.004	0.864 ± 0.006	0.891 ± 0.007	0.837 ± 0.009	0.939 ± 0.003
	Random Forest	0.872 ± 0.004	0.869 ± 0.006	0.889 ± 0.009	0.851 ± 0.007	0.940 ± 0.001
Easy B (41 feat)	KNN	0.855 ± 0.002	0.846 ± 0.005	0.901 ± 0.008	0.797 ± 0.007	0.923 ± 0.001

	Logistic Reg.	0.866 ± 0.003	0.862 ± 0.005	0.891 ± 0.008	0.834 ± 0.006	0.937 ± 0.003
	Random Forest	0.868 ± 0.004	0.865 ± 0.005	0.889 ± 0.008	0.842 ± 0.005	0.937 ± 0.003
Harder A (32 feat)	KNN	0.727 ± 0.004	0.711 ± 0.006	0.755 ± 0.010	0.673 ± 0.010	0.786 ± 0.007
	Logistic Reg.	0.720 ± 0.008	0.705 ± 0.007	0.747 ± 0.007	0.667 ± 0.010	0.777 ± 0.006
	Random Forest	0.745 ± 0.009	0.728 ± 0.011	0.779 ± 0.014	0.684 ± 0.012	0.815 ± 0.008
Harder B (26 feat)	KNN	0.645 ± 0.004	0.623 ± 0.005	0.665 ± 0.010	0.586 ± 0.010	0.699 ± 0.004
	Logistic Reg.	0.665 ± 0.005	0.608 ± 0.007	0.732 ± 0.008	0.521 ± 0.013	0.711 ± 0.006
	Random Forest	0.670 ± 0.005	0.598 ± 0.021	0.766 ± 0.041	0.493 ± 0.045	0.727 ± 0.010

Values shown are from a 20,000-row sample run; accuracy differs slightly (< 1.5pp) from the full-dataset values reported in Table 3. Relative ordering of models and the direction of all effects are preserved.

Appendix C — WordNet Relationship Types Between Definitions and Answers

Of the 127,608 unique definition–answer pairs, 44.1% are connected by at least one WordNet relationship within two hops, rising to about 52% at three hops. The table below describes the relationship types that appear most frequently.

Relationship	% of unique pairs	Definition	Example
Hyponym of hypernym (co-hyponymy)	~20%	Definition and answer share a common parent in the WordNet taxonomy	definition "dog" ↔ answer "cat" (both are hyponyms of animal)

Hyponym	~15%	The answer is a more specific instance of the definition	definition "bird" → answer "robin"
Synonym	~12%	Definition and answer share at least one WordNet synset	definition "quick" ↔ answer "fast"
Hypernym	(lower)	The answer is a more general category than the definition	definition "sedan" → answer "car"
Part meronym / holonym	rare	Part-whole relationship between definition and answer	definition "wheel" ↔ answer "car"

Percentages are of the 127,608 unique definition–answer pairs in the filtered dataset. Relationship types overlap; a single pair may satisfy several at once.

Appendix D — WordNet Relationship Types Between Definitions and Answers

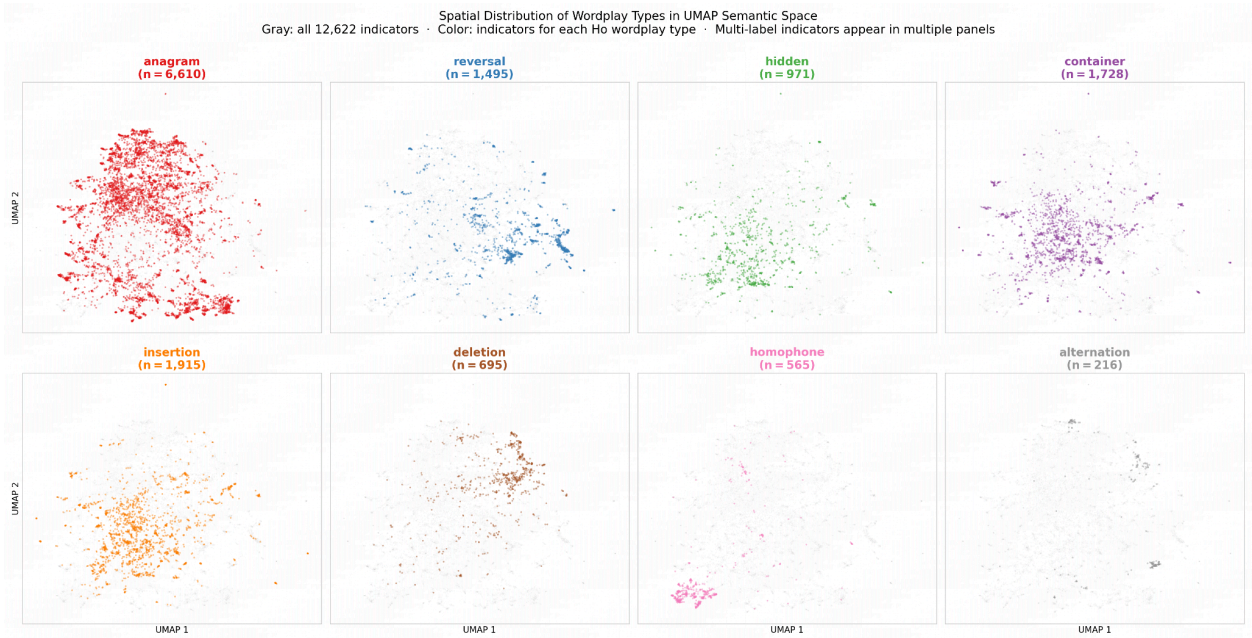

Figure D.1: 2D UMAP projection of the 12,622 indicator embeddings, shown as eight small-multiples panels. Each panel colours only the indicators labelled with one Ho wordplay type against the grey backdrop of all indicators; multi-label indicators appear in more than one panel. Anagram (n = 6,610) is by far the most populous type and covers nearly the full embedding space. The smaller types occupy more localized regions: homophone (n = 565) concentrates in the lower left, deletion (n = 695) in the upper centre, reversal and hidden overlap in the centre-right. The heavy spatial overlap between types is consistent with the finding in Part B that a single HDBSCAN or agglomerative clustering does not cleanly recover the eight-type taxonomy from semantics alone.